MODULE 05 · DOMAIN 1 - PLAN AND MANAGE AN AZURE AI SOLUTION

CI/CD for Foundry projects

Infrastructure and agents as code, separate environments, evaluation gates, pipeline identity and safe promotion.

DOMAIN 1 - Plan and manage WEIGHT 25-30% MODULE 05 of 39

admin@sthithapragna.com

Put **everything** in source control, and let an **evaluation**, not someone's impression, decide whether a change ships.

Everything that changes behaviour is code

ARTEFACT	WHY VERSION IT
Infrastructure: resource, projects, deployments	Rebuild any environment identically
Agent definition: instructions, model, tools	A prompt edit is a behaviour change
Hosted agent code and container	The normal build and test path
Evaluation datasets and thresholds	The gate itself must be reviewable
Per-environment configuration	Same code, different settings

ON THE EXAM

Secrets never go in the repository. Pipelines authenticate with Microsoft Entra ID instead.

How to separate dev, test and production

EXPLORATION

Share, to move fast

- A shared Foundry resource for experiments
- Non-production data only
- Low quotas and limited risk

PRODUCTION

Isolate by default

- Separate Foundry resources per environment
- Separate resource groups or subscriptions
- Deployed from the same templates
- Different parameters per environment

THE TIEBREAKER

Microsoft's rollout guidance: production defaults to isolation. A dev project inside the production resource is not isolation.

The azd AI agent extension

- **azure.yaml** declares the project, model deployments, connections, toolboxes and the agent.
- **azd up** provisions resources, deploys models, builds the container and publishes the agent.
- **azd environments** hold settings for dev, test and production. Built for CI/CD.
- **azd ai agent init --infra** writes Bicep, or Terraform, that you can customise and check in.
- **azd down** deletes every resource in the resource group. Never point it at production.

> ON THE EXAM
>
> 'Manage infrastructure and agents together, from the terminal and in pipelines' is azd's own description of when to use it.

Bicep templates for Foundry

- The Foundry resource: **Microsoft.CognitiveServices/accounts**, of kind AIServices.
- Projects, as child resources of that account.
- Model deployments: **accounts/deployments**, with the deployment type as the SKU name.
- One template, with a parameter file per environment: names, regions, capacity, SKUs.

> ON THE EXAM
>
> Deployment types from module 04 appear here as SKU names, such as GlobalStandard or DataZoneStandard.

A Foundry release pipeline

01
Commit
Code, prompts, agent definition, datasets.

02
Build and test
Unit tests, config checks, container build.

03
Evaluate
Quality and safety gate on a dataset.

04
Dev
Deploy, then quick smoke tests.

05
Test
Scenario and safety evaluations, then approval.

06
Production
Approved release, then monitoring.

ON THE EXAM

Promote the same tested version through each stage. Never rebuild or re-edit between test and production.

Evaluation in GitHub Actions and Azure DevOps

- A **GitHub Action** and an **Azure DevOps task** run offline evaluations of Foundry agents.
- You supply a **dataset of test queries** and a **list of evaluators**, plus the agent and its version.
- Evaluators come from the Foundry catalogue: task adherence, fluency, safety checks such as violence.
- Results include **confidence intervals and significance tests**, not just raw scores.
- Evaluators use a **judge model** deployment, so every run consumes tokens.

> ON THE EXAM
>
> The gate evaluates a specific agent version. That version is what gets promoted if it passes.

A score two points lower may be **noise**. A good gate asks whether a change is **significantly** worse, not just different.

Four kinds of check, four jobs

CHECK	CATCHES	EXAMPLE
Smoke test	An agent that deploys but does not respond	Send one prompt, expect an answer
Quality evaluation	Worse answers after a change	Task adherence pass rate of at least 85%
Safety evaluation	New harmful or unsafe output	Violence and other risk evaluators
Performance	Slower or more expensive responses	Latency and token counts per query

ON THE EXAM

Smoke tests are fast and cheap; evaluations are thorough and cost tokens. Run smoke tests first, and fail fast.

How the pipeline signs in, and what it may do

- **No stored secrets:** OpenID Connect federation from GitHub, or an Azure Resource Manager service connection.
- **Control plane** role to provision infrastructure, such as Contributor on the resource group.
- **Data plane** role to create agents and run evaluations, such as Foundry User on the project.
- Owner and Contributor **cannot** invoke agents or run evaluations by themselves.

> THE SIGNAL
>
> 'Provisioning succeeds, evaluation fails with an authorisation error' means the data plane role is missing.

Agent versions, and model versions

- Agent Service **snapshots a version** with each change. You can compare versions and roll back.
- **Publishing** promotes an agent version to a stable endpoint for applications to call.
- A new **model version** is a release too: evaluate it in test before switching production.
- Limit production to **generally available** features. Preview features can be hidden with tags or custom roles.

> ON THE EXAM
>
> Rollback should mean pointing back to a known-good version, not re-editing a prompt under pressure.

> A support team edits its production agent's instructions directly in the Foundry portal. Last week, one edit quietly made hundreds of answers worse before anyone noticed.
>
> Leadership wants every change tested automatically before it reaches customers, and a named person to approve each production release.

What should the team implement?

A. Require two developers to review each prompt edit in the portal before saving.

B. Put the agent in source control, with an evaluation gate and approval.

C. Evaluate the production agent every month, and roll back when scores fall.

D. Deploy every commit straight to production and rely on monitoring alerts.

◆ PAUSE AND DECIDE

What should the team implement?

A. Two reviewers in the portal.
Adds human eyes but no automated testing, and edits still land directly in production.

B. Source control, evaluation gate, approval.
Automated testing before release, and a named approver at the production stage.

C. Monthly production evaluation.
Detects damage weeks after customers see it. Testing must happen before release.

D. Straight to production with alerts.
No testing and no approval. Monitoring finds problems only after they ship.

THE SIGNAL

Two requirements, two mechanisms: 'tested automatically before' is a gate, 'a named person approves' is an approval stage.

> A pipeline uses a federated identity with the Contributor role on the resource group. Its Bicep step creates the Foundry resource, project and model deployments successfully.
>
> The next step runs an agent evaluation, and fails with an authorisation error when it tries to invoke the agent.

What should the team change?

A Grant the pipeline identity the Foundry User role on the project.

B Grant the pipeline identity the Owner role on the whole subscription.

C Store the resource's API key as a pipeline variable and use key authentication.

D Grant the pipeline identity the Foundry Account Owner role on the resource.

◆ PAUSE AND DECIDE

What should the team change?

A Foundry User on the project.

Adds the missing data plane permission to invoke agents and run evaluations, at a narrow scope.

B Owner on the subscription.

Hugely over-privileged, and Owner still has no data plane actions for building in a project.

C API key in a pipeline variable.

Keys bypass role-based access and add a secret to manage, undoing the federated sign-in.

D Foundry Account Owner.

A management role. It creates projects and models but cannot build or invoke in a project.

THE SIGNAL

Provisioning worked and invoking failed: control plane fine, data plane missing.

A team's evaluation gate fails any build whose average groundedness score is lower than the previous run's, by any amount. Its dataset has twelve queries.

Developers report that builds with no changes to prompts or code fail at random, and they have started rerunning pipelines until they pass.

How should the team improve the gate?

A. Remove the evaluation gate and rely on peer review of prompt changes.

B. Allow up to three automatic reruns, and pass the build if any run succeeds.

C. Grow the dataset, set a threshold, and fail only on significant drops.

D. Cut the dataset to five queries so each run is faster and cheaper.

◆ PAUSE AND DECIDE

How should the team improve the gate?

A Remove the gate.

Throws away the only automated protection against regressions.

B Pass if any of three reruns succeeds.

Formalises retrying until lucky, which lets real regressions through.

C Larger dataset, threshold, significant drops only.

Separates real regressions from normal run-to-run variation.

D Five queries.

Fewer queries make scores noisier, so random failures increase.

THE SIGNAL

Random failures with no changes are the fingerprint of a gate that treats noise as signal.

A start-up runs development, testing and its live customer agent in a single Foundry project, sharing one model deployment.

Last Tuesday, a developer's load test used up the deployment's quota, and customers saw errors for an hour. Test agents have also reached production data.

What should the start-up do?

A Keep one project, and schedule all developer testing outside business hours.

B Create separate test agents with a -dev suffix inside the same project.

C Request a much higher tokens-per-minute quota for the shared model deployment.

D Separate Foundry resources for production and non-production, from templates.

◆ PAUSE AND DECIDE

What should the start-up do?

A Test outside business hours.

Customers use services at night too, and test agents still reach production data.

B Separate agents in the same project.

Same deployment, same quota, and a shared identity. A new name isolates nothing.

C Raise the shared quota.

Delays the next outage, and does nothing about test agents reaching production data.

D Separate resources from shared templates.

Isolates quota, identity and data, while templates keep environments consistent.

THE TIEBREAKER

Two failures, quota and data access, share one root cause: no environment boundary. Fix the boundary.

CI/CD for Foundry in five lines

- **Everything that changes behaviour is code:** infrastructure, agents, prompts, datasets.
- **Separate resources per environment**, from the same templates. Production isolated.
- **Evaluation gates** in GitHub Actions or Azure DevOps. Fail on significant drops, not noise.
- **Pipeline identity needs both planes:** Contributor to provision, Foundry User to evaluate.
- **Promote the tested version**; roll back to a known-good version; treat model upgrades as releases.

> ON THE EXAM
>
> Next, module 06: quotas, rate limits, scaling and cost.